

Git

GitHub Commit & Push

Complete Guide

For Beginners

[Git Basics](#)

[Terminal Commands](#)

[PAT Auth](#)

[Commit & Push](#)

[Troubleshooting](#)

00 Git Terminology

Term	Description
git	Code version management tool (like a time machine for your code)
GitHub	Website to store and share Git-managed code online
Repository	Project folder / storage on GitHub (called 'repo')
commit	Save button. Records changes as a snapshot
push	Upload commits from your PC to GitHub
pull	Download latest code from GitHub to your PC
branch	Workspace. Default is 'main'
remote	Connected remote repository (GitHub URL)
clone	Copy an entire GitHub repo to your PC
PAT	Personal Access Token. Auth key instead of password

01 Initial Setup (One - Time Only)

Git Installation Check

```
Terminal
$ git --version
git version 2.43.0 # Installed!
```

User Configuration

```
Terminal - Run Once
# Set your name and email (same as GitHub account)
$ git config --global user.name "Your Name"
$ git config --global user.email "you@email.com"
```

PAT (Personal Access Token) Setup

GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic)
> Generate new token > Check 'repo' scope > Generate token
> Copy the ghp_xxxxxxxx token and save it securely!

Warning: The token is shown ONLY ONCE. Save it before closing the page!

02 Starting a New Project

Method A: Local Project > Push to GitHub

STEP 1. Create an empty repo on GitHub (DO NOT check 'Add README')

STEP 2. Copy the repo URL (e.g. <https://github.com/myid/my-project.git>)

```
Terminal – Project Init & Connect

# Move to your project folder
$ cd /path/to/my-project

# Initialize Git
$ git init

# Set branch name to main
$ git branch -M main

# Connect to GitHub
$ git remote add origin https://github.com/myid/my-project.git
```

```
Terminal – First Commit & Push

# Stage all files
$ git add .

# Create first commit
$ git commit -m "Initial commit: project setup"

# Upload to GitHub
$ git push -u origin main
Username: your-github-id
Password: (paste your PAT token here)
```

TIP: Password input is invisible (nothing shows when typing). Just paste and press Enter!

Method B: Clone from GitHub

```
Terminal – Clone

# Clone repo to your PC (auto-connects remote)
$ git clone https://github.com/myid/my-project.git
$ cd my-project
```

03 Daily Workflow: Commit & Push

The Big 3 — These 3 commands are all you need!

```
Terminal — The Big 3 (Daily Commands)

$ git add .                # 1. Stage all changes
$ git commit -m "your message" # 2. Save (commit)
$ git push                 # 3. Upload to GitHub
```

Detailed Steps

STEP 1. Check what changed

```
Terminal — Status Check

$ git status
modified:   App.java      # Red = not staged yet
new file:   Utils.java    # Green = ready to commit
```

STEP 2. Stage files (add)

```
Terminal — Staging

$ git add .                # Add ALL changed files
$ git add src/App.java     # Or add specific file
```

STEP 3. Commit (save)

```
Terminal — Commit

$ git commit -m "feat: add login feature"
```

STEP 4. Push (upload)

```
Terminal — Push

$ git push
```

04 Commit Message Convention

Format: type: description

Type	Meaning	Example
feat	New feature	feat: add bus arrival notification
fix	Bug fix	fix: resolve map display error
docs	Documentation	docs: update README
style	Design/UI change	style: change theme to navy
refactor	Code cleanup	refactor: remove duplicate code
test	Add tests	test: add login validation test
chore	Build/config	chore: update Maven dependencies

05 .gitignore

Create a .gitignore file in the project root to exclude files from GitHub.

```
.gitignore

# Build output
target/
build/
*.jar
*.class

# IDE settings
.idea/
.vscode/

# OS files
.DS_Store

# Secrets (NEVER upload these!)
.env
```

WARNING: NEVER commit .env files, PAT tokens, or passwords. Add them to .gitignore!

06 Useful Commands

Status & History

```
Terminal - Status & Log

$ git status          # Check current changes
$ git log --oneline -10  # View last 10 commits
$ git diff            # See file changes in detail
```

Undo / Revert

```
Terminal - Undo

# Discard changes in a file (before commit)
$ git checkout -- filename.java

# Undo last commit (keep file changes)
$ git reset --soft HEAD~1

# Unstage a file (undo git add)
$ git reset HEAD filename.java

# Fix last commit message
$ git commit --amend -m "corrected message"
```

Branch (Workspace)

```
Terminal - Branch

# Create and switch to new branch
$ git checkout -b feature/login

# List all branches
$ git branch

# Switch back to main
$ git checkout main
```

Pull (Download Latest)

```
Terminal - Pull

# Download latest code from GitHub
$ git pull
```

07 Common Mistakes & Fixes

1. Authentication Error (password removed)

```
remote: Support for password authentication was removed
```

Fix: Use PAT token instead of password (see Section 01)

2. Push Rejected (fetch first)

```
! [rejected] main -> main (fetch first)
```

Fix: Run: `git pull - -rebase origin main` then `git push`

3. Committed Sensitive Data (tokens/passwords)

```
Accidentally pushed PAT or API keys
```

Fix: 1. Revoke the token immediately 2. Generate new one 3. Add file to `.gitignore`

4. Added Wrong File

```
git add included unwanted files
```

Fix: Run: `git reset HEAD filename` to unstage

5. Typo in Commit Message

```
Commit message has errors
```

Fix: Run: `git commit - -amend - m "corrected message"`
(before push only!)

08 Flow Diagram & Cheatsheet

[Local (Your PC)]

```
git init
git remote add origin URL
Edit Code...
git add .
git commit -m "message"
git push
git pull
```

----->

=====>

<=====

[GitHub (Remote)]

Connect

Upload

Download

Daily Git Cheatsheet

```
# Start of day – get latest code
git pull

# After editing – check what changed
git status

# Save & Upload (the big 3)
git add .
git commit -m "feat: your changes"
git push

# Check recent history
git log --oneline -5

# Undo last commit (keep files)
git reset --soft HEAD~1
```

TIP: git add . > git commit > git push
Make this a daily habit and you'll never lose your code!